

MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications

Andrew G. Howard Menglong Zhu Bo Chen Dmitry Kalenichenko
Weijun Wang Tobias Weyand Marco Andreetto Hartwig Adam

Google Inc.

{howarda,menglong,bochen,dkalenichenko,weijunw,weyand,anm,hadam}@google.com

轻量级神经网络“巡礼”（二）—— MobileNet，从V1到V3



R.JD

计算机视觉研发/人工智能教育

关注他

来自专栏 · 深度学习需要深度学习 >

2052 人赞同了该文章 >

自从2017年由谷歌公司提出，MobileNet可谓是轻量级网络中的Inception，经历了一代又一代的更新。成为了学习轻量级网络的必经之路。

MobileNet V1

MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications

Andrew G. Howard Menglong Zhu Bo Chen Dmitry Kalenichenko
Weijun Wang Tobias Weyand Marco Andreetto Hartwig Adam

Google Inc.

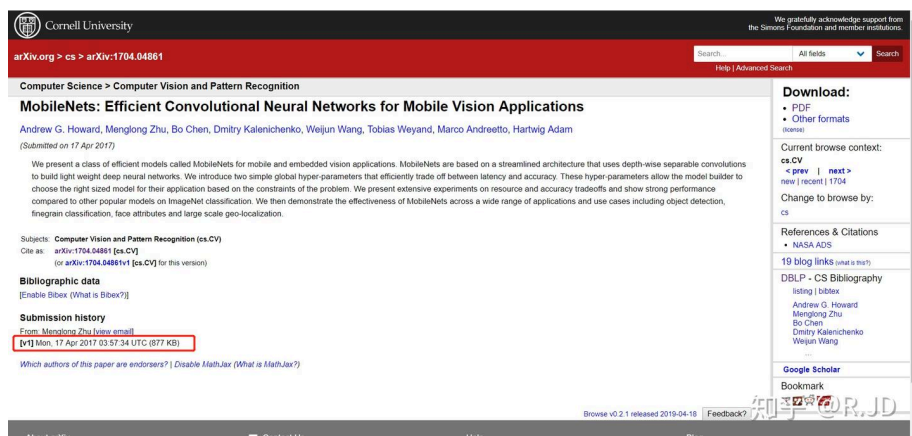
{howarda,menglong,bochen,dkalenichenko,weijunw,weyand,anm,hadam}@google.com

知乎 @R.JD

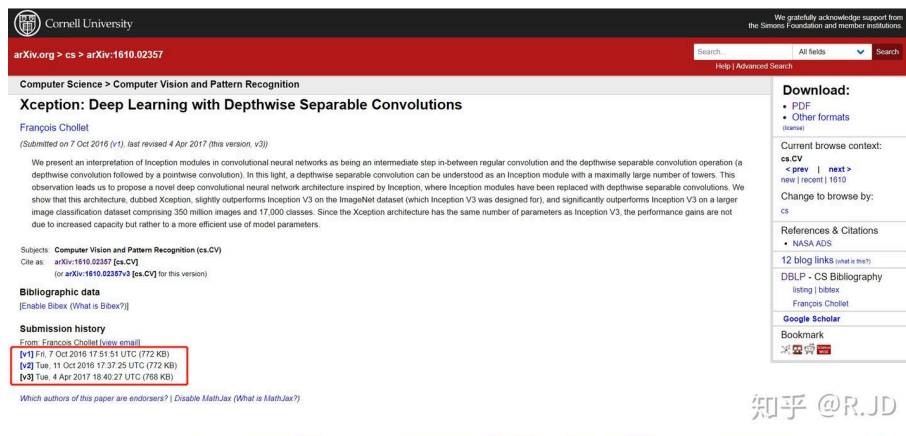
MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications 论文地址：arxiv.org/abs/1704.04861 收录：CVPR2017

2017年4月，谷歌提出MobileNetV1，这一专注于在移动设备上的轻量级神经网络。一直都有一个争议，说MobileNetV1怎么和Xception的网络block结构一样，都大量用到了深度可分离？

其实这里有个小插曲： MobileNetV1在arxiv.org上的介绍是：



可以看到MobileNetV1是在2017年4月才提交的v1版本，但是



Xception是早在2016年10月v1版本就提出来了。那么，真的是MobileNet“抄袭”了Xception吗？其实并不是的，在Xception v1版本论文当中有这样的一句话：

Sifre developed separable convolutions during an internship at Google Brain in 2013, and used them in AlexNet to obtain small gains in accuracy and large gains in convergence speed, as well as a significant reduction in model size. An overview of his work was first made public in a presentation at ICLR 2014 [21]. Detailed experimental results are reported in Sifre's thesis, section 6.2 [13]. This initial work on separable convolutions was inspired by prior research from Sifre and Mallat on transformation-invariant scattering [14, 13]. Later, a separable convolution was used as the first layer of Inception V1 and Inception V2 [18, 6]. Within Google, Andrew Howard has investigated how separable convolutions could be used to build small networks that retained much of the capabilities of Inception V2 or V3, with applications in mobile computing. Additionally, our work is only possible due to the inclusion of separable convolutions in the TensorFlow framework [1].

- Residual connections, introduced by He et al. in [4], which our proposed architecture uses extensively.

而这个Andrew Howard是谁呢？没错，就是MobileNetV1的作者。而在Xception v3论文版本中，一句话就变成了：

sis, section 6.2 [15]. This initial work on depthwise separable convolutions was inspired by prior research from Sifre and Mallat on transformation-invariant scattering [16, 15]. Later, a depthwise separable convolution was used as the first layer of Inception V1 and Inception V2 [20, 7]. Within Google, Andrew Howard [6] has introduced efficient mobile models called MobileNets using depthwise separable convolutions. Jin et al. in 2014 [8] and Wang et al. in 2016 [24] also did related work aiming at reducing the size and computational cost of convolutional neural networks using separable convolutions. Additionally, our work is only possible due to the inclusion of an efficient implementation of

真相只有一个：2016年6月，谷歌提出了MobileNetV1，由于各种原因当时没有挂上arxiv，一直到2017年4月才提交。好巧不巧，谷歌的另一团队，同时提出了Xception。所有才有了两个结构都是基于深度可分离卷积*的相似争论。

好了，键盘党们可以擦擦键盘休息休息了。

废话说完，开始正文（好像每篇文章都有开头废话啊喂！！）

其实介绍MobileNetV1（以下简称V1）只有一句话，**MobileNetV1就是把VGG中的标准卷积层换成深度可分离卷积就可以了。**

那么，这个**深度可分离卷积**是什么？

深度可分离卷积

深度可分离卷积（*depthwise separable convolution*），根据史料记载，可追溯到2012年的论文[Simplifying ConvNets for Fast Learning](#)，作者提出了可分离卷积的概念（下图（a））：

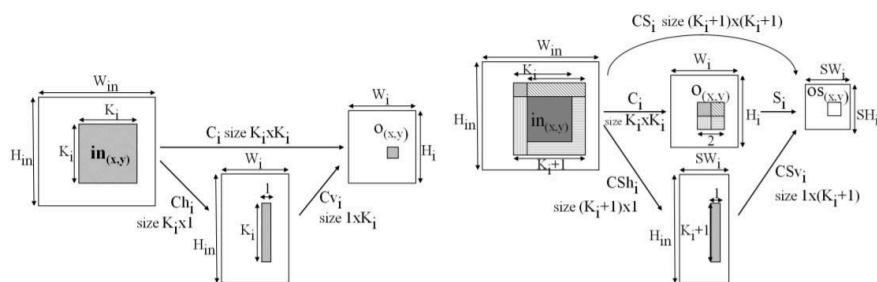


Fig. 2. (a) Separable convolution layers; (b) Fused separable convolution and sub-sampling layers.

Laurent Sifre博士2013年在谷歌实习期间，将可分离卷积拓展到了深度（depth），并且在他的博士论文[Rigid-motion scattering for image classification](#)中有详细的描写，感兴趣的同学可以去看看论文

可分离卷积主要有两种类型：**空间可分离卷积**和**深度可分离卷积**。

空间可分离

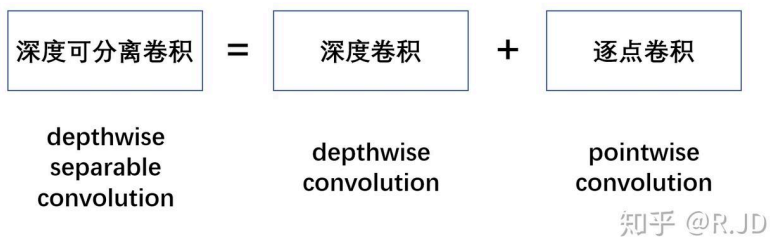
顾名思义，空间可分离就是将一个大的卷积核变成两个小的卷积核，比如将一个 3×3 的核分成一个 3×1 和一个 1×3 的核：

$$\begin{bmatrix} 1 & 2 & 3 \\ 0 & 0 & 0 \\ 2 & 4 & 6 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 2 \end{bmatrix} \times \begin{bmatrix} 1 & 2 & 3 \end{bmatrix}$$

由于空间可分离卷积不在MobileNet的范围内，就不说了。

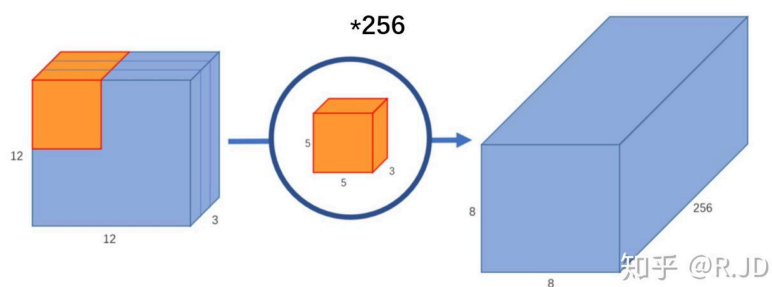
深度可分离卷积

MobileNet基于深度可分离卷积



深度可分离卷积就是将普通卷积拆分成为一个深度卷积和一个逐点卷积。

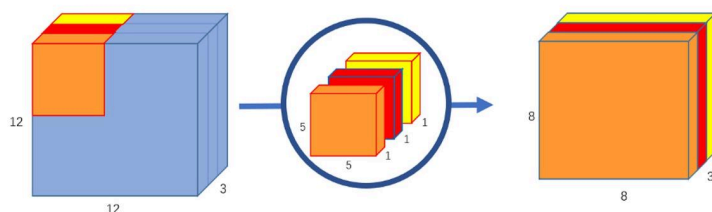
我们先来看一下标准的卷积操作：



输入一个 $12 \times 12 \times 3$ 的一个输入特征图，经过 $5 \times 5 \times 3$ 的卷积核卷积得到一个 $8 \times 8 \times 1$ 的输出特征图。如果此时我们有256个特征图，我们将会得到一个 $8 \times 8 \times 256$ 的输出特征图。

以上就是标准卷积做干的活。那深度卷积和逐点卷积呢？

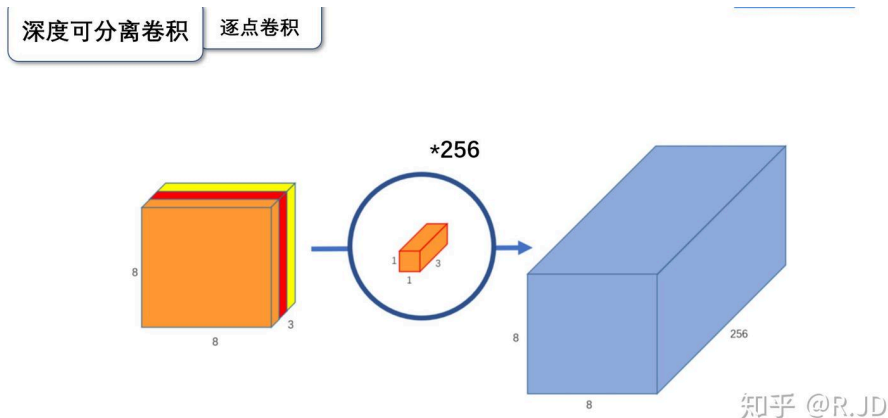
深度卷积



与标准卷积网络不一样的是，我们将卷积核拆分成但单通道形式，在不改变输入特征图像的深度的情况下，对每一通道进行卷积操作，这样就得到了和输入特征图通道数一致的输出特征图。如上图：输入 $12 \times 12 \times 3$ 的特征图，经过 $5 \times 5 \times 1 \times 3$ 的深度卷积之后，得到了 $8 \times 8 \times 3$ 的输出特征图。输入个输出的维度是不变的3。这样就会有一个问题，通道数太少，特征图的维度太少，能获取到足够的有效信息吗？

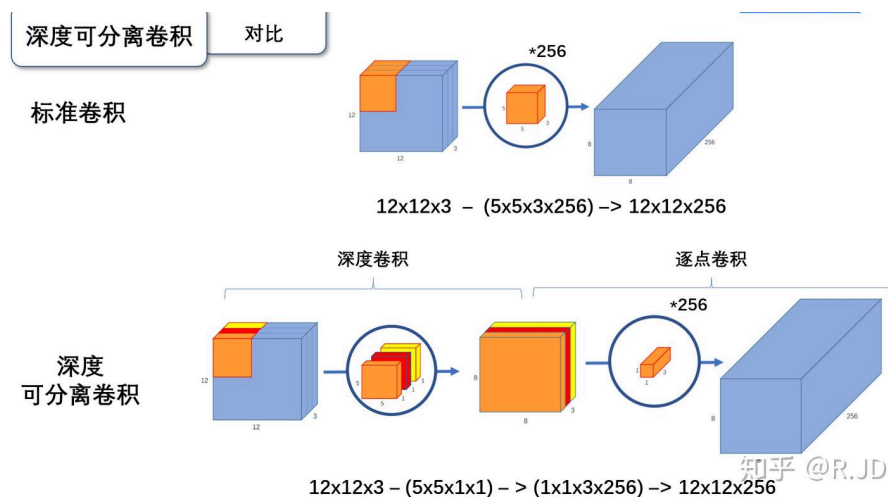
逐点卷积

逐点卷积就是 1×1 卷积。主要作用就是对特征图进行升维和降维，如下图：



在深度卷积的过程中，我们得到了 $8 \times 8 \times 3$ 的输出特征图，我们用256个 $1 \times 1 \times 3$ 的卷积核对输入特征图进行卷积操作，输出的特征图和标准的卷积操作一样都是 $8 \times 8 \times 256$ 了。

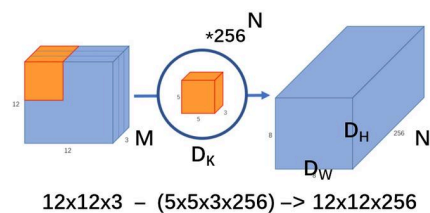
标准卷积与深度可分离卷积的过程对比如下：



为什么要深度可分离卷积？

这个问题很好回答，如果有一个方法能让你用更少的参数，更少的运算，但是能达到差的不是很多的结果，你会使用吗？

深度可分离卷积就是这样的一个方法。我们首先来计算一下标准卷积的参数量与计算量（只考虑MAdd）：



$$\text{Params} \quad D_K \times D_K \times M \times N$$

$$\text{计算量} \quad D_K \times D_K \times M \times N \times D_W \times D_H$$

标准卷积的参数量

卷积核的尺寸是 $D_K \times D_K \times M$ ，一共有 N 个，所以标准卷积的参数量是：

$$\text{Params} \quad D_K \times D_K \times M \times N$$

标准卷积的计算量

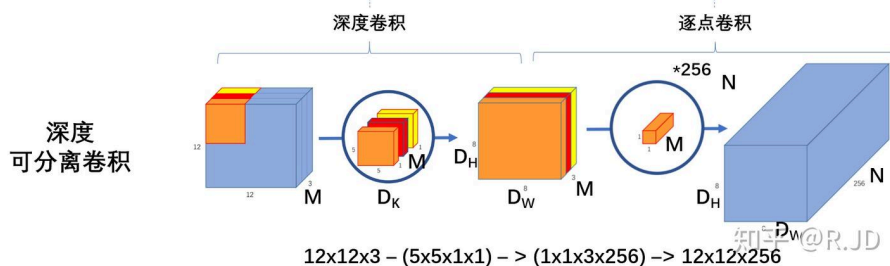
卷积核的尺寸是 $D_K \times D_K \times M$ ，一共有 N 个，每一个都要进行 $D_W \times D_H$ 次运算，所以标准卷积的计算量是：

$$\text{计算量} \quad D_K \times D_K \times M \times N \times D_W \times D_H$$

标准卷积算完了，我们接下来计算深度可分离卷积的参数量和计算量：

$$\text{Params} \quad D_K \times D_K \times M + M \times N$$

$$\text{计算量} \quad D_K \times D_K \times M \times D_W \times D_H + M \times N \times D_W \times D_H$$



深度可分离卷积的参数量

深度可分离卷积的参数量由深度卷积和逐点卷积两部分组成：

深度卷积的卷积核尺寸 $D_k \times D_k \times M$ ；逐点卷积的卷积核尺寸为 $1 \times 1 \times M$ ，一共有 N 个，所以深度可分离卷积的参数数量是：

Params
$$D_K \times D_K \times M + M \times N$$

深度可分离卷积的计算量

深度可分离卷积的计算量也是由深度卷积和逐点卷积两部分组成：

深度卷积的卷积核尺寸 $D_k \times D_k \times M$ ，一共要做 $D_w \times D_h$ 次乘加运算；逐点卷积的卷积核尺寸为 $1 \times 1 \times M$ ，有 N 个，一共要做 $D_w \times D_h$ 次乘加运算，所以深度可分离卷积的计算量是：

计算量
$$D_K \times D_K \times M \times D_W \times D_H + M \times N \times D_W \times D_H$$

总的来说：

Params		
深度可分离卷积	$D_K \times D_K \times M + M \times N$	
标准卷积	$D_K \times D_K \times M \times N$	
		$= \frac{1}{N} + \frac{1}{D_K^2}$
计算量		
深度可分离卷积	$D_K \times D_K \times M \times D_W \times D_H + M \times N \times D_W \times D_H$	
标准卷积	$D_K \times D_K \times M \times N \times D_W \times D_H$	

可以参数数量和乘加操作的运算量均下降为原来的

$$\frac{1}{N} + \frac{1}{D_K^2}$$

我们通常所使用的是 3×3 的卷积核，也就是会下降到原来的九分之一到八分之一。

假设

输出为一个 $224 \times 224 \times 3$ 的图像，VGG网络某层卷积输入的尺寸是 $112 \times 112 \times 64$ 的特征图，卷积核为 $3 \times 3 \times 128$ ，标准卷积的运算量是：

$$3 \times 3 \times 128 \times 64 \times 112 \times 112 = 924844032$$

深度可分离卷积的运算量是：

$$3 \times 3 \times 64 \times 112 \times 112 + 128 \times 64 \times 112 \times 112 = 109985792$$

这一层，MobileNetV1所采用的深度可分离卷积计算量与标准卷积计算量的比值为：

$$109985792 / 924844032 = 0.1189$$

与我们所计算的九分之一到八分之一一致。

V1卷积层

V1网络结构

标准卷积 VS V1卷积块

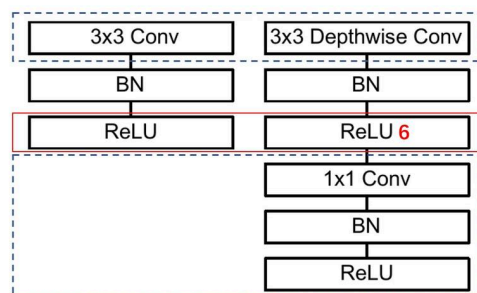
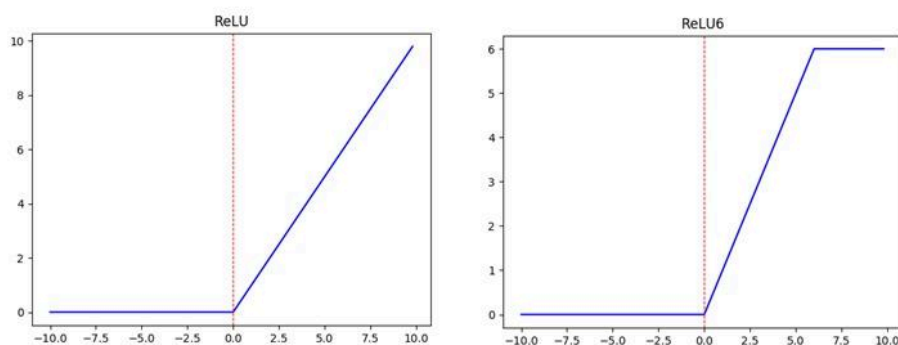


Figure 3. Left: Standard convolutional layer with batchnorm and ReLU. Right: Depthwise Separable convolutions with Depthwise and Pointwise layers followed by batchnorm and ReLU.

上图左边是标准卷积层，右边是V1的卷积层，虚线处是不相同点。V1的卷积层，首先使用 3×3 的深度卷积提取特征，接着是一个BN层，随后是一个ReLU层，在之后就会逐点卷积，最后就是BN和ReLU了。这也很符合深度可分离卷积，**将左边的标准卷积拆分成右边的一个深度卷积和一个逐点卷积。**

等等，我们发现有什么东西混了进来？？ReLU6+是什么？

ReLU6



$$ReLU(6) = \min(\max(0, x), 6)$$

上图左边是普通的ReLU，对于大于0的值不进行处理，右边是ReLU6，当输入的值大于6的时候，返回6，relu6“具有一个边界”。作者认为ReLU6作为非线性激活函数，在低精度计算下具有更强的鲁棒性。（这里所说的“低精度”，我看到有人说不是指的float16，而是指的定点运算(fixed-point arithmetic)）

现在就有一个问题，标准卷积核深度可分离卷积层到底对结果有什么样的影响呢？

上实验。

V1网络结构

标准卷积层VS V1卷积层

Table 4. Depthwise Separable vs Full Convolution MobileNet

Model	ImageNet Accuracy	Million Mult-Adds	Million Parameters
Conv MobileNet	71.7%	4866	29.3
MobileNet	70.6%	569	4.2

知乎 @R.JD

可以看到使用深度可分离卷积与标准卷积，参数和计算量能下降为后者的九分之一到八分之一左右。但是准确率只有下降极小的1%。

V1网络结构

V1网络结构

Table 1. MobileNet Body Architecture

Type / Stride	Filter Shape	Input Size
Conv / s2	$3 \times 3 \times 3 \times 32$	$224 \times 224 \times 3$
Conv dw / s1	$3 \times 3 \times 32 \text{ dw}$	$112 \times 112 \times 32$
Conv / s1	$1 \times 1 \times 32 \times 64$	$112 \times 112 \times 32$
Conv dw / s2	$3 \times 3 \times 64 \text{ dw}$	$112 \times 112 \times 64$
Conv / s1	$1 \times 1 \times 64 \times 128$	$56 \times 56 \times 64$
Conv dw / s1	$3 \times 3 \times 128 \text{ dw}$	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 128$	$56 \times 56 \times 128$
Conv dw / s2	$3 \times 3 \times 128 \text{ dw}$	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 256$	$28 \times 28 \times 128$
Conv dw / s1	$3 \times 3 \times 256 \text{ dw}$	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 256$	$28 \times 28 \times 256$
Conv dw / s2	$3 \times 3 \times 256 \text{ dw}$	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 512$	$14 \times 14 \times 256$
5×	Conv dw / s1	$3 \times 3 \times 512 \text{ dw}$
	Conv / s1	$1 \times 1 \times 512 \times 512$
	Conv dw / s2	$3 \times 3 \times 512 \text{ dw}$
Conv / s1	$1 \times 1 \times 512 \times 1024$	$7 \times 7 \times 512$
Conv dw / s2	$3 \times 3 \times 1024 \text{ dw}$	$7 \times 7 \times 1024$
Conv / s1	$1 \times 1 \times 1024 \times 1024$	$7 \times 7 \times 1024$
Avg Pool / s1	Pool 7×7	$7 \times 7 \times 1024$
FC / s1	1024×1000	$1 \times 1 \times 1024$
Softmax / s1	Classifier	$1 \times 1 \times 1000$

知乎 @R.JD

MobileNet的网络结构如上图所示。首先是一个3x3的标准卷积，s2进行下采样。然后就是堆积深度可分离卷积，并且其中的部分深度卷积会利用s2进行下采样。然后采用平均池化层将feature变成1x1，根据预测类别大小加上全连接层，最后是一个softmax层。整个网络有28层，其中深度卷积层有13层。

实验结果

V1论文中还有一部分对V1网络再进行调整，在此就不赘述了，感兴趣的同学可以去看看原文。

V1的效果到底好不好，作者将V1与大型网络GoogleNet和VGG16进行了比较：

Table 8. MobileNet Comparison to Popular Models

Model	ImageNet Accuracy	Million Mult-Adds	Million Parameters
1.0 MobileNet-224	70.6%	569	4.2
GoogleNet	69.8%	1550	6.8
VGG 16	71.5%	15300	138

知乎 @R.JD

可以发现，作为轻量级网络的V1在计算量小于GoogleNet，参数量差不多是在一个数量级的基础上，在分类效果上比GoogleNet还要好，这就是要得益于深度可分离卷积了。VGG16的计算量参数量比V1大了30倍，但是结果也仅仅只高了1%不到。

目标检测，在COCO数据集上的结果：

Table 13. COCO object detection results comparison using different frameworks and network architectures. mAP is reported with COCO primary challenge metric (AP at IoU=0.50:0.05:0.95)

Framework Resolution	Model	mAP	Billion Mult-Adds	Million Parameters
SSD 300	deeplab-VGG	21.1%	34.9	33.1
	Inception V2	22.0%	3.8	13.7
	MobileNet	19.3%	1.2	6.8
Faster-RCNN 300	VGG	22.9%	64.3	138.5
	Inception V2	15.4%	118.2	13.3
	MobileNet	16.4%	25.2	6.1
Faster-RCNN 600	VGG	25.7%	149.6	138.5
	Inception V2	21.9%	129.6	13.3
	MobileNet	19.8%	30.5	6.1

知乎 @R.JD

对了，作者还在论文中分析整个网络的参数和计算量分布，如下图所示。可以看到整个计算量基本集中在1x1卷积上。对于参数也主要集中在1x1卷积，除此之外还有就是全连接层占了一部分参数。

Table 2. Resource Per Layer Type

Type	Mult-Adds	Parameters
Conv 1×1	94.86%	74.59%
Conv DW 3×3	3.06%	1.06%
Conv 3×3	1.19%	0.02%
Fully Connected	0.18%	24.33%

知乎 @R.JD

MobileNetV2: Inverted Residuals and Linear Bottlenecks

Mark Sandler Andrew Howard Menglong Zhu Andrey Zhmoginov Liang-Chieh Chen
Google Inc.
{sandler, howarda, menglong, azhmogin, lcchen}@google.com

知乎 @R.JD

MobileNetV2: Inverted Residuals and Linear Bottlenecks 论文地址:

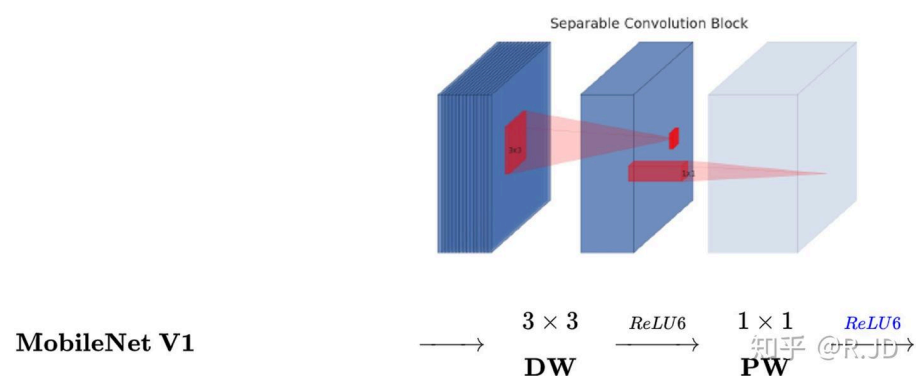
arxiv.org/abs/1704.04861 收录: CVPR2018

MobileNetV1 (以下简称: V1) 过后, 我们就要讨论讨论MobileNetV2 (以下简称: V2) 了。为了能更好地讨论V2, 我们首先再回顾一下V1:

回顾MobileNet V1

V1核心思想是采用 **深度可分离卷积** 操作。在相同的权值参数数量的情况下, 相较标准卷积操作, 可以减少数倍的计算量, 从而达到提升网络运算速度的目的。

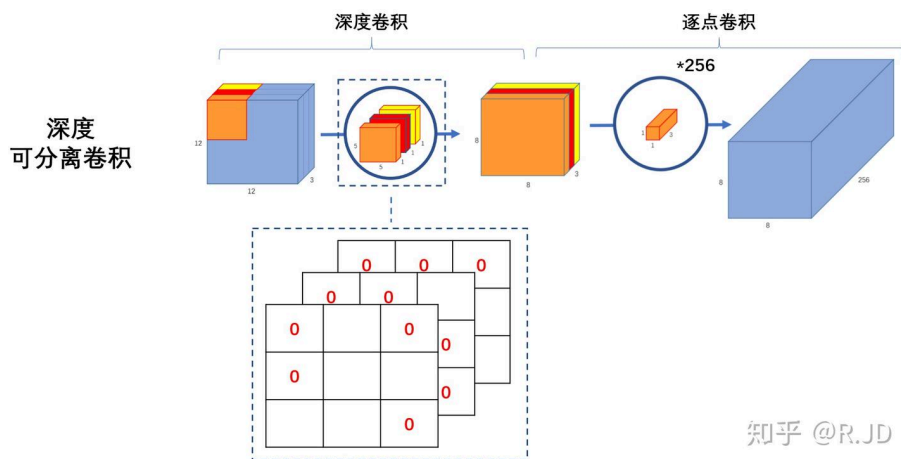
V1的block如下图所示:



首先利用 3×3 的深度可分离卷积提取特征, 然后利用 1×1 的卷积来扩张通道。用这样的block堆叠起来的MobileNetV1既能较少不小的参数量、计算量, 提高网络运算速度, 又能得到接近于标准卷积的还不错的结果, 看起来是很美好的。

但是!

有人在实际使用的时候, 发现深度卷积部分的卷积核比较容易训废掉: 训完之后发现深度卷积训出来的卷积核有不少是空的:



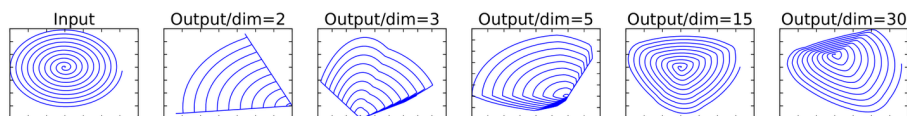
这是为什么？

作者认为这是**ReLU**这个浓眉大眼的激活函数的锅。（没想到你个浓眉大眼的ReLU激活函数也叛变革命了？？？）

ReLU做了些啥？

V2的论文中，作者也有这样的解释。（论文中的实在不是很好懂，我就根据一些解读结合我的想法简单说说吧。有说的不正确的地方，还请各位大佬指出，感谢！）

这是将低维流形的ReLU变换embedded到高维空间中的的例子。



我们在这里抛弃掉流形这个概念，通俗理解一下。

假设在2维空间有一组m个点组成的螺旋线 X_m 数据(如input)，利用随机矩阵 T 映射到 n 维空间上并进行ReLU运算，即：

$$y = \text{ReLU}(T \cdot X_m)$$

其中， X_m 被随机矩阵 T 映射到了 n 维空间：

$$y = \text{ReLU}(T \cdot X_m)$$

n维空间

再利用随机矩阵T的逆矩阵T-1，将y映射回2维空间当中：

$$X'_m = T^{-1} \cdot y$$

全过程如下表示：

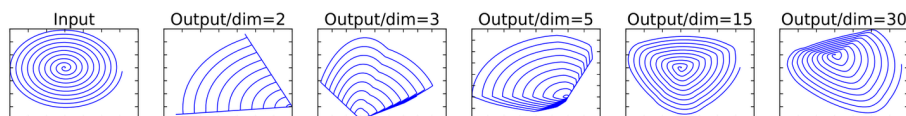
$$X_m^{-1} = T^{-1} \left(\text{ReLU}(T \cdot X_m) \right)$$

n维空间

y

换句话说，就是对一个n维空间中的一个“东西”做ReLU运算，然后（利用T的逆矩阵T-1恢复）对比ReLU之后的结果与Input的结果相差有多大。

可以看到：



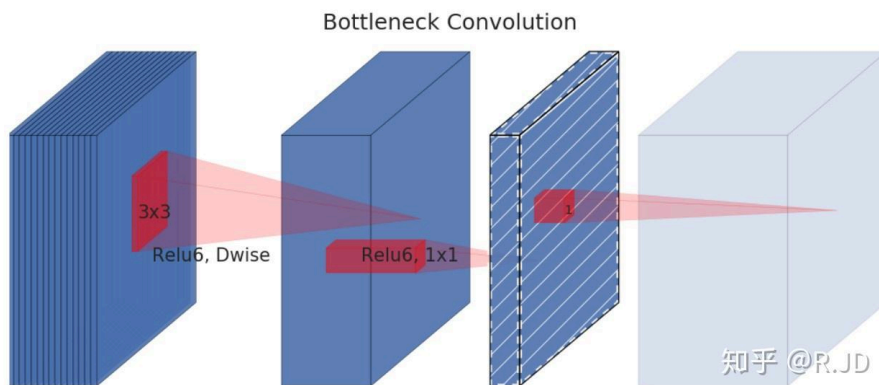
当n = 2,3时，与Input相比有很大一部分的信息已经丢失了。而当n = 15到30，还是有相当多的地方被保留了下来。

也就是说，对低维度做ReLU运算，很容易造成信息的丢失。而在高维度进行ReLU运算的话，信息的丢失则会很少。

这就解释了为什么深度卷积的卷积核有不少是空。发现了问题，我们就能更好地解决问题。针对这个问题，可以这样解决：既然是ReLU导致的信息损耗，将ReLU替换成线性激活函数。

Linear bottleneck

我们当然不能把所有的激活层都换成线性的啊，所以我们就悄咪咪的把最后的那个ReLU6换成Linear。（至于为什么换最后一个ReLU6而不换第一个和第二个ReLU6，看到后面就知道了。）

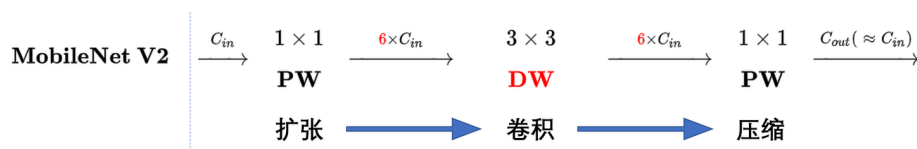


Separable with linear bottleneck

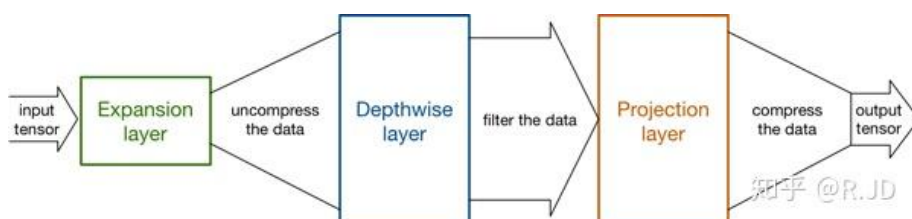
作者将这个部分称之为**linear bottleneck**。对，就是论文名中的那个linear bottleneck。

Expansion layer

现在还有个问题是，深度卷积本身没有改变通道的能力，来的是多少通道输出就是多少通道。如果来的通道很少的话，DW深度卷积只能在低维度上工作，这样效果并不会很好，所以我们要“**扩张**”通道。既然我们已经知道PW逐点卷积也就是 1×1 卷积可以用来升维和降维，那就可以在DW深度卷积之前使用PW卷积进行升维（升维倍数为 t ， $t=6$ ），再在一个更高维的空间中进行卷积操作来提取特征：

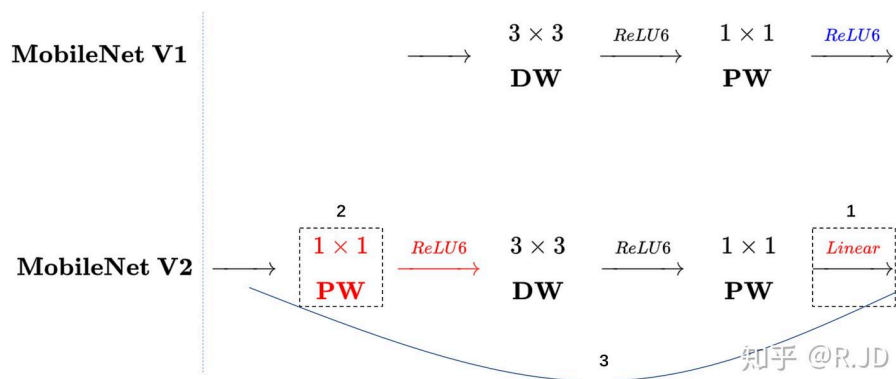


也就是说，不管输入通道数是多少，经过第一个PW逐点卷积升维之后，深度卷积都是在相对的更高6倍维度上进行工作。

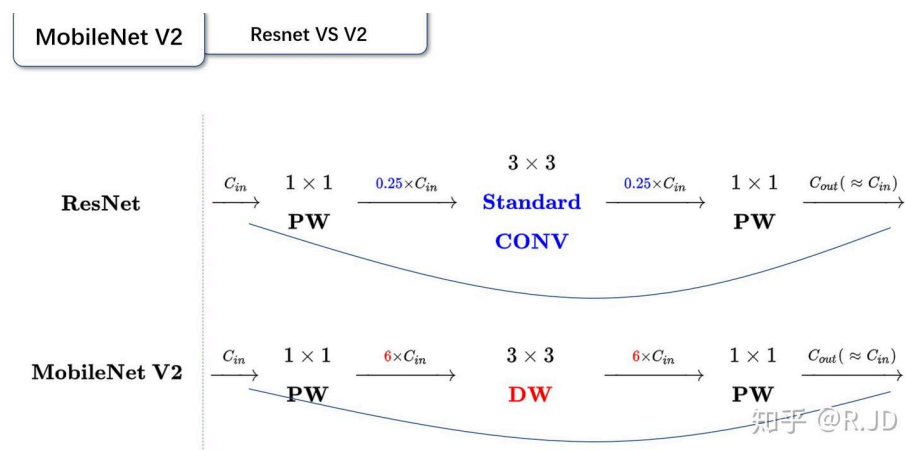


Inverted residuals⁺

回顾V1的网络结构，我们发现V1很像是一个直筒型的VGG网络。我们想像Resnet一样复用我们的特征，所以我们引入了**shortcut结构**，这样V2的block就是如下图形式：



对比一下V1和V2:



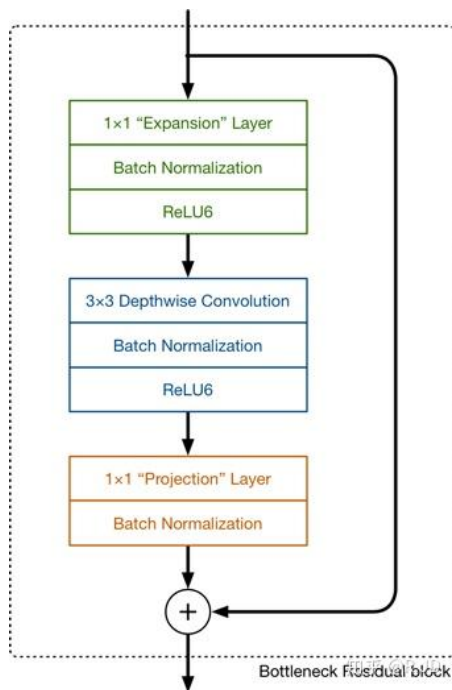
可以发现，都采用了 $1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$ 的模式，以及都使用Shortcut结构。但是不同点呢：

- ResNet 先降维 (0.25倍)、卷积、再升维。
- MobileNetV2 则是 先升维 (6倍)、卷积、再降维。

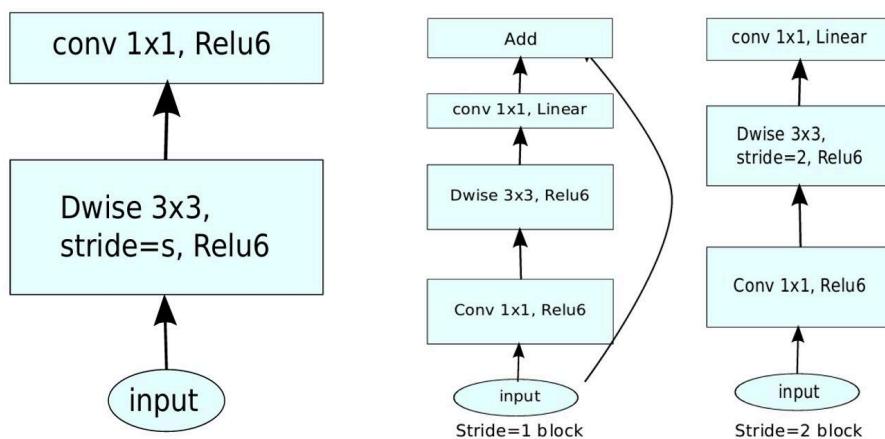
刚好V2的block刚好与Resnet的block相反，作者将其命名为**Inverted residuals**。就是论文名中的**Inverted residuals**。

V2的block

至此，V2的最大的创新点就结束了，我们再总结一下V2的block:



我们将V1和V2的block进行一下对比：



(b) MobileNet[27]

(d) Mobilenet V2

左边是v1的block，没有Shortcut并且带最后的ReLU6。

右边是v2的加入了 1×1 升维，引入Shortcut并且去掉了最后的ReLU，改为Linear。步长为1时，先进行 1×1 卷积升维，再进行深度卷积提取特征，再通过Linear的逐点卷积降维。将input与output相加，形成残差结构。步长为2时，因为input与output的尺寸不符，因此不添加shortcut结构，其余均一致。

V2的网络结构

Input	Operator	t	c	n	s
$224^2 \times 3$	conv2d	-	32	1	2
$112^2 \times 32$	bottleneck	1	16	1	1
$112^2 \times 16$	bottleneck	6	24	2	2
$56^2 \times 24$	bottleneck	6	32	3	2
$28^2 \times 32$	bottleneck	6	64	4	2
$14^2 \times 64$	bottleneck	6	96	3	1
$14^2 \times 96$	bottleneck	6	160	3	2
$7^2 \times 160$	bottleneck	6	320	1	1
$7^2 \times 320$	conv2d 1x1	-	1280	1	1
$7^2 \times 1280$	avgpool 7x7	-	-	1	-
$1 \times 1 \times 1280$	conv2d 1x1	-	k	-	-

t 是输入通道的倍增系数（即扩张倍数）

n 是该模块重复次数

c 是输出通道数

s 是stride（后面重复都是stride 1）

知乎 @R.JD

$28 \times 28 \times 32$ 那一层的步长为2的话，输出应该是 14×14 ，应该是一处错误。按照作者论文里的说法，自己修改了一下：

Input	Operator	t	c	n	s
$224^2 \times 3$	conv2d	-	32	1	2
$112^2 \times 32$	bottleneck	1	16	1	1
$112^2 \times 16$	bottleneck	6	24	2	2
$56^2 \times 24$	bottleneck	6	32	3	2
$28^2 \times 32$	bottleneck	6	64	4	1
$14^2 \times 64$	bottleneck	6	96	3	2
$14^2 \times 96$	bottleneck	6	160	3	2
$7^2 \times 160$	bottleneck	6	320	1	1
$7^2 \times 320$	conv2d 1x1	-	1280	1	1
$7^2 \times 1280$	avgpool 7x7	-	-	1	-
$1 \times 1 \times 1280$	conv2d 1x1	-	k	-	-

t 是输入通道的倍增系数（即扩张倍数）

n 是该模块重复次数

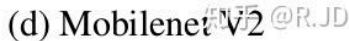
c 是输出通道数

s 是stride（后面重复都是stride 1）

知乎 @R.JD

实验结果

Image Classification



结果如下：

Table 4: Performance on ImageNet, comparison for different networks. As is common practice for ops, we count the total number of Multiply-Adds. In the last column we report running time in milliseconds (ms) for a single large core of the Google Pixel 1 phone (using TF-Lite). We do not report ShuffleNet numbers as efficient group convolutions and shuffling are not yet supported.

详细对比如下：

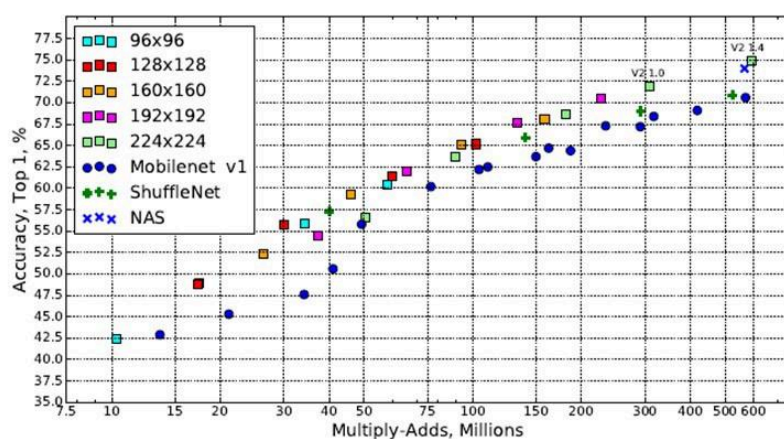


Figure 5: Performance curve of MobileNetV2 vs MobileNetV1, ShuffleNet, NAS. For our networks we use multipliers 0.35, 0.5, 0.75, 1.0 for all resolutions, and additional 1.4 for for 224. Best viewed in color.

Object Detection

SSDLite

目标检测方面，作者首先提出了SSDLite。就是对SSD结构做了修改，将SSD的预测层中所有标准卷积替换为深度可分离卷积。作者说这样参数量和计算成本大大降低，计算更高效。SSD与SSDLite对比：

	Params	MAdds
SSD[33]	14.8M	1.25B
SSDLite	2.1M	0.35B

Table 5: Comparison of the size and the computational cost between SSD and SSDLite configured with MobileNetV2 and making predictions for 80 classes.

应用在物体检测任务上，V1与常用检测网络的对比：

Network	mAP	Params	MAdd	CPU
SSD300[34]	23.2	36.1M	35.2B	-
SSD512[34]	26.8	36.1M	99.5B	-
YOLOv2[35]	21.6	50.7M	17.5B	-
MNet V1 + SSDLite	22.2	5.1M	1.3B	270ms
MNet V2 + SSDLite	22.1	4.3M	0.8B	200ms

Table 6: Performance comparison of MobileNetV2 + SSDLite and other realtime detectors on the COCO dataset object detection task. MobileNetV2 + SSDLite achieves competitive accuracy with significantly fewer parameters and smaller computational complexity. All models are trained on `trainval35k` and evaluated on `test-dev`. SSD/YOLOv2 numbers are from [35]. The running time is reported for the large core of the Google Pixel 1 phone, using an internal version of the TF-Lite engine.

知乎 @R.JD

可以看到，基于MobileNetV2的SSDLite在COCO数据集上超过了YOLOv2，并且大小小10倍速度快20倍。

Semantic Segmentation

Network	OS	ASPP	MF	mIOU	Params	MAdds
MNet V1	16	✓		75.29	11.15M	14.25B
	8	✓	✓	78.56	11.15M	941.9B
MNet V2*	16	✓		75.70	4.52M	5.8B
	8	✓	✓	78.42	4.52M	387B
MNet V2*	16			75.32	2.11M	2.75B
	8		✓	77.33	2.11M	152.6B
ResNet-101	16	✓		80.49	58.16M	81.0B
	8	✓	✓	82.70	58.16M	4870.6B

Table 7: MobileNet + DeepLabv3 inference strategy on the PASCAL VOC 2012 *validation* set. **MNet V2***: Second last feature map is used for DeepLabv3 heads, which includes (1) Atrous Spatial Pyramid Pooling (**ASPP**) module, and (2) 1×1 convolution as well as image-pooling feature. **OS**: *output_stride* that controls the output resolution of the segmentation map. **MF**: Multi-scale and left-right flipped inputs during test. All of the models have been pretrained on COCO. The potential candidate for on-device applications is shown in bold face. PASCAL images have dimension 512×512 and atrous convolution allows us to control output feature resolution without increasing the number of parameters.

知乎 @R.JD

分割效果如下：



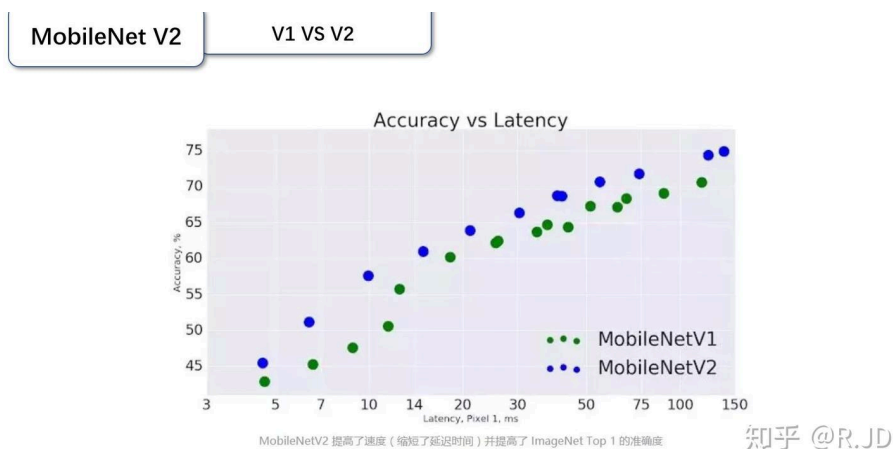
Figure 8: MobileNetV2 semantic segmentation visualization results on PASCAL VOC 2012 val set. **OS**: *output_stride*. **S**: single scale input. **MS+F**: Multi-scale inputs with scales = {0.5, 0.75, 1, 1.25, 1.5, 1.75} and left-right flipped inputs. Employing *output_stride* = 16 and single input scale = 1 attains a good trade-off between FLOPS and accuracy.

V1 VS V2

模型	网络层数(卷积)	FLOPs(M)	Params(M)	CPU耗时(ms)
MobileNetV1	28(13DW)	575	4.2	120
MobileNetV2	54(17DW)	300	3.4	80

可以看到，虽然V2的层数比V1的要多很多，但是FLOPs，参数以及CPU耗时都是比V1要好的。

V1V2在google Pixel 1手机上在Image Classification任务的对比：



MobileNetV2 模型在整体速度范围内可以更快实现相同的准确性。

目标检测和语义分割的结果：

MobileNet V2

V1 VS V2

目标检测

模型	参数(M)	Madds(B)	mAP(%)	移动CPU(ms)
MobileNetV1+SSDLite	5.1	1.3	22.2	270
MobileNetV1+SSDLite	4.3	0.8	22.1	200

语义分割

模型	参数(M)	Madds(B)	mIOU(%)
MobileNetV1+DeepLabV3	11.15	14.25	75.29
MobileNetV1+DeepLabV3	2.11	2.75	75.32

知乎 @R.JD

综上，MobileNetV2 提供了一个非常高效的面向移动设备的模型，可以用作许多视觉识别任务的基础。

但是！

在我实际应用V1V2时，V1的效果都要稍微好一点。上一张gluonCV的结果图，和我的实现也差不多：

MobileNet

HINT

- MobileNet1.0_int8 is a quantized model for MobileNet1.0.

Model	Top-1	Top-5	Hashtag	Training Command	Training Log
MobileNet1.0 [4]	73.28	91.30	efbb2ca3	shell script	log
MobileNet1.0_int8 [4]	72.85	90.99	efbb2ca3		
MobileNet1.0 (no mixup) [4]	72.93	91.14	cce75496	shell script	log
MobileNet0.75 [4]	70.25	89.49	84c801e2	shell script	log
MobileNet0.5 [4]	65.20	86.34	0130d2aa	shell script	log
MobileNet0.25 [4]	52.91	76.94	f0046a3d	shell script	log
MobileNetV2_1.0 [5]	72.04	90.57	f9952bcd	shell script	log
MobileNetV2_0.75 [5]	69.36	88.50	b56e3d1c	shell script	log
MobileNetV2_0.5 [5]	64.43	85.31	08038185	shell script	log
MobileNetV2_0.25 [5]	51.76	74.89	9b1d2cc3	shell script	log

知乎 @R.JD

不知道为什么。

MobileNet V3

V1, V2都看完了，现在就来到了MobileNetV3（以下简称V3）。

Searching for MobileNetV3

Andrew Howard¹ Mark Sandler¹ Grace Chu¹ Liang-Chieh Chen¹ Bo Chen¹ Mingxing Tan²
Weijun Wang¹ Yukun Zhu¹ Ruoming Pang² Vijay Vasudevan² Quoc V. Le² Hartwig Adam¹
¹Google AI, ²Google Brain
{howarda, sandler, cxy, lcchen, bochen, tanmingxing, weijunw, yukun, rpang, vrv, qvl, hadam}@google.com

知乎 @R.JD

Searching for MobileNetV3 论文地址: [arxiv.org/pdf/1905.0224...](https://arxiv.org/pdf/1905.02244v1.pdf)

MobileNetV3, 是谷歌在2019年3月21日提出的网络架构。首先, 引入眼帘的是这篇文章的标题, “searching” 一词就把V3的论文的核心观点展示了出来——用**神经结构搜索 (NAS⁺)**来完成V3。虽然本人没有接触过NAS, 但是我已经闻到了金钱的味道。

"抱歉, 有钱真的可以为..."

由于真的没有接触过NAS, 所以V3就讲讲其他的, 除NAS之外的东西吧。

先上结果:

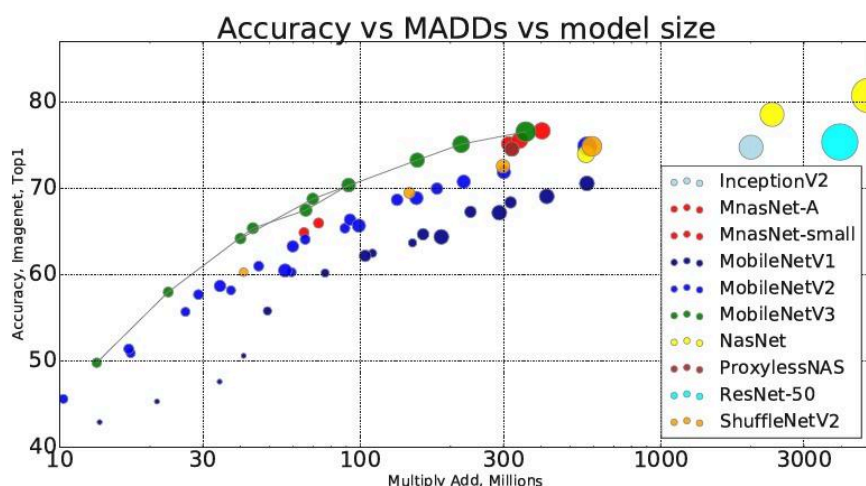


Figure 2. The trade-off between MADDs and top-1 accuracy. This allows to compare models that were targeted different hardware or software frameworks. All MobileNetV3 are for input resolution 224 and use multipliers 0.35, 0.5, 0.75, 1 and 1.25. See section 6 for other resolutions. Best viewed in color.

知乎 @R.JD

可以看到, 在同样大小的计算量下, V3在ImageNet上的结果都是最好的。

我们先来看看V3做了什么?

MobileNetV3的相关技术

- 0. 网络的架构基于NAS实现的MnasNet (效果比MobileNetV2好)
- 1. 引入MobileNetV1的深度可分离卷积
- 2. 引入MobileNetV2的具有线性瓶颈的倒残差结构
- 3. 引入基于squeeze and excitation结构的轻量级注意力模型(SE)
- 4. 使用了一种新的激活函数h-swish⁺(x)

- 5.网络结构搜索中，结合两种技术：资源受限的NAS（platform-aware NAS）与 NetAdapt
- 6.修改了MobileNetV2网络端部最后阶段

第0点，关于MnasNet也是基于NAS的，也不是很了解。大家感兴趣的话，可以参考曲晓峰老师的这个回答[如何评价 Google 最新的模型 MnasNet? - 曲晓峰的回答 - 知乎](#)，写的很棒！所以我们只要认为MnasNet是一个比MobileNet精度和实时性更高的模型就行了。

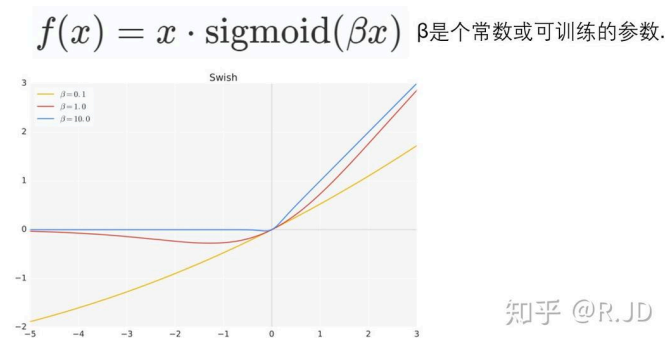
第1,2点在前面的MobileNetV1和V2上有讨论，在这里就不赘述了。

第3点引入SE模块，主要为了利用结合特征通道的关系来加强网络的学习能力。先不仔细讨论，之后在【深度回顾经典网络】系列的时候再详细讨论吧，感兴趣的同学，可以看看这一篇文章。

激活函数h-swish

swish

h-swish是基于swish的改进，swish最早是在谷歌大脑2017的论文[Searching for Activation functions](#)所提出（又是Searching for!! ！）。



swish论文的作者认为，Swish具备无上界有下界、平滑、非单调的特性。并且Swish在深层模型上的效果优于ReLU。仅仅使用Swish单元替换ReLU就能把MobileNet,NASNetA在ImageNet上的top-1分类准确率提高0.9%，Inception-ResNet-v的分类准确率提高0.6%。

V3也利用swish当作为ReLU的替代时，它可以显著提高神经网络的精度。但是呢，作者认为这种非线性激活函数虽然提高了精度，但在嵌入式环境中，是有不少的成本的。原因就是在移动设备上计算sigmoid函数是非常明智的选择。所以提出了h-swish。

h-swish

可以用一个近似函数来逼近这个swish，让swish变得硬(hard)。作者选择的是基于ReLU6，作者认为几乎所有的软件和硬件框架上都可以使用ReLU6的优化实现。其次，它能在特定模式下消除了由于近似sigmoid的不同实现而带来的潜在的数值精度损失。

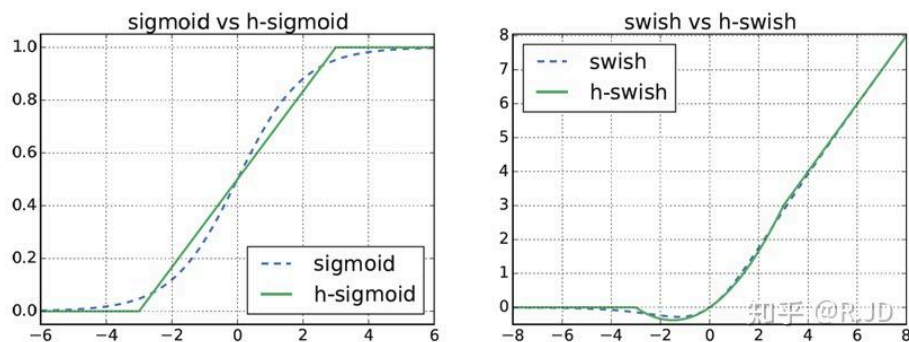
$$f(x) = x \cdot \text{sigmoid}(\beta x) \quad \leftarrow \quad \frac{\text{ReLU6}(x+3)}{6}$$

$$\downarrow$$

$$\text{h-swish}[x] = x \frac{\text{ReLU6}(x+3)}{6}$$

知乎 @R.JD

下图是Sigmoid和swish的hard、soft形式：



我们可以简单的认为，hard形式是soft形式的低精度化。作者认为swish的表现和其他非线性相比，能够将过滤器的数量减少到16个的同时保持与使用ReLU或swish的32个过滤器相同的精度，这节省了3毫秒的时间和1000万MAdds的计算量。

并且同时，作者认为随着网络的深入，应用非线性激活函数的成本会降低，能够更好的减少参数量。作者发现swish的大多数好处都是通过在更深的层中使用它们实现的。因此，在V3的架构中，只在模型的后半部分使用h-swish(HS)。

网络结构搜索NAS

由于不熟，就简单写一点吧。

主要结合两种技术：资源受限的NAS（platform-aware NAS）与NetAdapt。

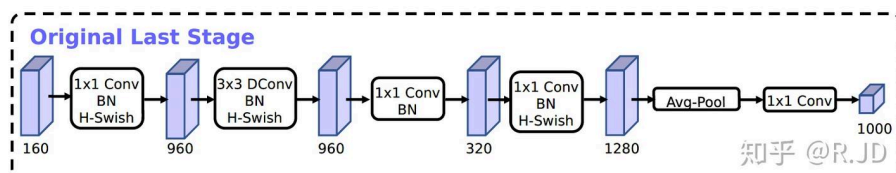
资源受限的NAS，用于在计算和参数量受限的前提下搜索网络来优化各个块（block），所以称之为模块级搜索（Block-wise Search）。

NetAdapt，用于对各个模块确定之后网络层的微调每一层的卷积核数量，所以称之为层级搜索（Layer-wise Search）。

一旦通过体系结构搜索找到模型，我们就会发现最后一些层以及一些早期层计算代价比较高昂。于是作者决定对这些架构进行一些修改，以减少这些慢层(slow layers)的延迟，同时保持准确性。这些修改显然超出了当前搜索的范围。

对V2最后阶段的修改

作者认为，当前模型是基于V2模型中的倒残差结构和相应的变体（如下图）。使用 1×1 卷积来构建最后层，这样可以便于拓展到更高维的特征空间。这样做的好处是，在预测时，有更多更丰富的特征来满足预测，但是同时也引入了额外的计算成本与延时。



所以，需要改进的地方就是要保留高维特征的前提下减小延时。首先，还是将 1×1 层放在到最终平均池之后。这样的话最后一组特征现在不是 7×7 （下图V2结构红框），而是以 1×1 计算（下图V3结构黄框）。

Input	Operator	t	c	n	s
$224^2 \times 3$	conv2d	-	32	1	2
$112^2 \times 32$	bottleneck	1	16	1	1
$112^2 \times 16$	bottleneck	6	24	2	2
$56^2 \times 24$	bottleneck	6	32	3	2
$28^2 \times 32$	bottleneck	6	64	4	1
$14^2 \times 64$	bottleneck	6	96	3	2
$14^2 \times 96$	bottleneck	6	160	3	2
$7^2 \times 160$	bottleneck	6	320	1	1
$7^2 \times 320$	conv2d 1x1	-	1280	1	1
$7^2 \times 1280$	avgpool 7x7	-	-	1	-
$1 \times 1 \times 1280$	conv2d 1x1	-	k	-	-

t 是输入通道的倍增系数（即扩张倍数）

n 是该模块重复次数

c 是输出通道数

s 是stride（后面重复都是stride 1）

知乎 @R.JD

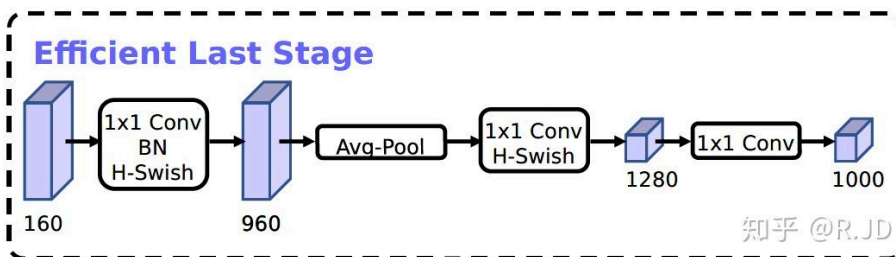
Input	Operator	exp size	#out	SE	NL	s
$224^2 \times 3$	conv2d	-	16	-	HS	2
$112^2 \times 16$	bneck, 3x3	16	16	-	RE	1
$112^2 \times 16$	bneck, 3x3	64	24	-	RE	2
$56^2 \times 24$	bneck, 3x3	72	24	-	RE	1
$56^2 \times 24$	bneck, 5x5	72	40	✓	RE	2
$28^2 \times 40$	bneck, 5x5	120	40	✓	RE	1
$28^2 \times 40$	bneck, 5x5	120	40	✓	RE	1
$28^2 \times 40$	bneck, 3x3	240	80	-	HS	2
$14^2 \times 80$	bneck, 3x3	200	80	-	HS	1
$14^2 \times 80$	bneck, 3x3	184	80	-	HS	1
$14^2 \times 80$	bneck, 3x3	184	80	-	HS	1
$14^2 \times 80$	bneck, 3x3	480	112	✓	HS	1
$14^2 \times 112$	bneck, 3x3	672	112	✓	HS	1
$14^2 \times 112$	bneck, 5x5	672	160	✓	HS	2
$7^2 \times 160$	bneck, 5x5	960	160	✓	HS	1
$7^2 \times 160$	bneck, 5x5	960	160	✓	HS	1
$7^2 \times 160$	conv2d, 1x1	-	960	-	HS	1
$7^2 \times 960$	pool, 7x7	-	-	-	-	-
$1^2 \times 960$	conv2d 1x1, NBN	-	1280	-	HS	1
$1^2 \times 1280$	conv2d 1x1, NBN	-	k	-	-	-

MobileNetV3-large

Input	Operator	exp size	#out
$224^2 \times 3$	conv2d, 3x3	-	16
$112^2 \times 16$	bneck, 3x3	16	16
$56^2 \times 16$	bneck, 3x3	72	24
$28^2 \times 24$	bneck, 3x3	88	24
$28^2 \times 24$	bneck, 5x5	96	40
$14^2 \times 40$	bneck, 5x5	240	40
$14^2 \times 40$	bneck, 5x5	240	40
$14^2 \times 40$	bneck, 5x5	120	48
$14^2 \times 48$	bneck, 5x5	144	48
$14^2 \times 48$	bneck, 5x5	288	96
$7^2 \times 96$	bneck, 5x5	576	96
$7^2 \times 96$	bneck, 5x5	576	96
$7^2 \times 96$	conv2d, 1x1	-	576
$7^2 \times 576$	pool, 7x7	-	-
$1^2 \times 576$	conv2d 1x1, NBN	-	1280
$1^2 \times 1280$	conv2d 1x1, NBN	-	k

MobileNetV3-small

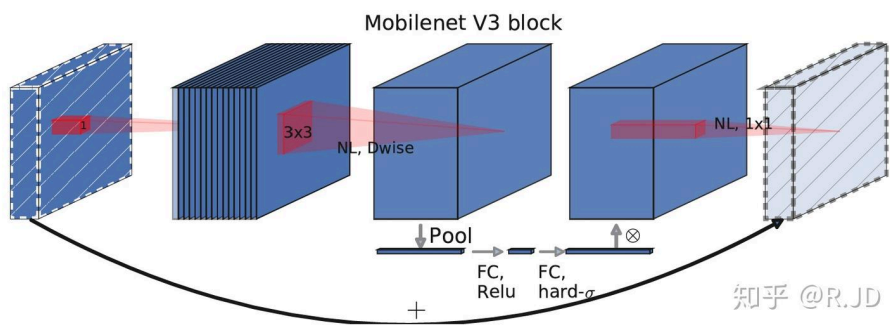
这样的好处是，在计算和延迟方面，特征的计算几乎是免费的。最终，重新设计完的结构如下：



在不会造成精度损失的同时，减少10ms耗时，提速15%，减小了30m的MAdd操作。

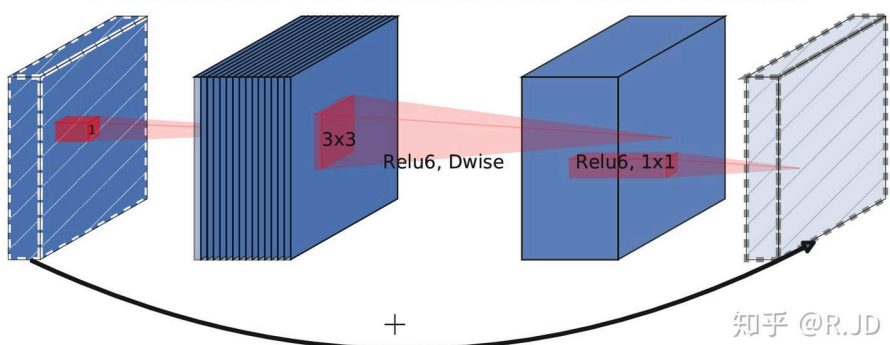
V3的block

综合以上，V3的block结构如下所示：



与V2的block相比较：

MobileNet V2: bottleneck with residual



MobileNetV3的网络结构

MobileNetV3定义了两个模型: **MobileNetV3-Large**和**MobileNetV3-Small**。V3-Large是针对**高资源**情况下的使用，相应的，V3-small就是针对**低资源**情况下的使用。两者都是基于之前的简单讨论的NAS。

MobileNetV3-Large

Input	Operator	exp size	#out	SE	NL	s
$224^2 \times 3$	conv2d	-	16	-	HS	2
$112^2 \times 16$	bneck, 3x3	16	16	-	RE	1
$112^2 \times 16$	bneck, 3x3	64	24	-	RE	2
$56^2 \times 24$	bneck, 3x3	72	24	-	RE	1
$56^2 \times 24$	bneck, 5x5	72	40	✓	RE	2
$28^2 \times 40$	bneck, 5x5	120	40	✓	RE	1
$28^2 \times 40$	bneck, 5x5	120	40	✓	RE	1
$28^2 \times 40$	bneck, 3x3	240	80	-	HS	2
$14^2 \times 80$	bneck, 3x3	200	80	-	HS	1
$14^2 \times 80$	bneck, 3x3	184	80	-	HS	1
$14^2 \times 80$	bneck, 3x3	184	80	-	HS	1
$14^2 \times 80$	bneck, 3x3	480	112	✓	HS	1
$14^2 \times 112$	bneck, 3x3	672	112	✓	HS	1
$14^2 \times 112$	bneck, 5x5	672	160	✓	HS	2
$7^2 \times 160$	bneck, 5x5	960	160	✓	HS	1
$7^2 \times 160$	bneck, 5x5	960	160	✓	HS	1
$7^2 \times 160$	conv2d, 1x1	-	960	-	HS	1
$7^2 \times 960$	pool, 7x7	-	-	-	-	1
$1^2 \times 960$	conv2d 1x1, NBN	-	1280	-	HS	1
$1^2 \times 1280$	conv2d 1x1, NBN	-	k	-	-	1

Table 1. Specification for MobileNetV3-Large. SE denotes whether there is a Squeeze-And-Excite in that block. NL denotes the type of nonlinearity used. Here, HS denotes h-swish and RE denotes ReLU. NBN denotes no batch normalization. *s* denotes stride.

知乎 @R.JD

MobileNetV3-Small

Input	Operator	exp size	#out	SE	NL	s
$224^2 \times 3$	conv2d, 3x3	-	16	-	HS	2
$112^2 \times 16$	bneck, 3x3	16	16	✓	RE	2
$56^2 \times 16$	bneck, 3x3	72	24	-	RE	2
$28^2 \times 24$	bneck, 3x3	88	24	-	RE	1
$28^2 \times 24$	bneck, 5x5	96	40	✓	HS	2
$14^2 \times 40$	bneck, 5x5	240	40	✓	HS	1
$14^2 \times 40$	bneck, 5x5	240	40	✓	HS	1
$14^2 \times 40$	bneck, 5x5	120	48	✓	HS	1
$14^2 \times 48$	bneck, 5x5	144	48	✓	HS	1
$14^2 \times 48$	bneck, 5x5	288	96	✓	HS	2
$7^2 \times 96$	bneck, 5x5	576	96	✓	HS	1
$7^2 \times 96$	bneck, 5x5	576	96	✓	HS	1
$7^2 \times 96$	conv2d, 1x1	-	576	✓	HS	1
$7^2 \times 576$	pool, 7x7	-	-	-	-	1
$1^2 \times 576$	conv2d 1x1, NBN	-	1280	-	HS	1
$1^2 \times 1280$	conv2d 1x1, NBN	-	k	-	-	1

Table 2. Specification for MobileNetV3-Small. See table 1 for notation.

知乎 @R.JD

就像之前所说的：只有在更深层次使用h-swish才能得到比较大的好处。所以在上面的网络模型中，不论大小，作者只在模型的后半部分使用h-swish。

用谷歌pixel 1/2/3来对大小V3进行测试的结果。

Network	Top-1	P-1	P-2	P-3
V3-Large 1.0	73.8	57	55.9	38.9
V2 1.0	70.9	62	53.6	38.4
V3-Small	64.9	21	20.5	13.9
V2 0.35	57.2	20.6	18.6	13.1

Table 4. Quantized performance. All latencies are in ms. The inference latency is measured using single large core on the respective Pixel 1/2/3 device. 知乎 @R.JD

实验结果

Image Classification

Network	Top-1	Madds	Params	P-1	P-2	P-3
V3-Large 1.0	75.2	219	5.4M	66	77	52.6
V3-Large 0.75	73.3	155	4M	50	61	40.1
Mnasnet-A1	75.2	315	3.9	84	93	64
Proxyless[5]	74.6	320	4M	88	98	65
V2 1.0	72.0	300	3.4M	78	90	61.3
V3-Small 1.0	67.4	66	2.9M	21.3	26	17.7
V3-Small 0.75	65.4	44	2.4M	18	21.06	15.1
Mnasnet [43]	64.9	65.1	1.9M	24.9	28.1	18.3
V2 0.35	60.8	59.2	1.6M	19	22	14.5

Table 3. Floating point performance on Pixel family of phones (“P-*n*“ denotes a Pixel-*n* phone). All latencies are in ms. The inference latency is measured using a single large core with a batch size of 1. 知乎 @R.JD

Detection

Backbone	mAP	Latency (ms)	Params (M)	MAdd (B)
V1	22.2	270	5.1	1.3
V2	22.1	200	4.3	0.80
Mnasnet	23.0	215	4.88	0.84
V3	22.0	173	4.97	0.62
V3[†]	22.0	150	3.22	0.51
V2 0.35	13.7	54.8	0.93	0.16
V2 0.5	16.6	80.4	1.54	0.27
Mnasnet 0.35	15.6	58.2	1.02	0.18
Mnasnet 0.5	18.5	86.2	1.68	0.29
V3-Small	16.0	67.2	2.49	0.21
V3-Small[†]	16.1	56	1.77	0.16

Table 6. Object detection results of SSDLite with different backbones on COCO test set. [†]: Channels in the blocks between C_4 and C_5 are reduced by a factor of 2. 知乎 @R.JD

Semantic Segmentation

N	Backbone	RF2	SH	F	mIOU	Params	Madds	CPU (f)	CPU (h)
1	V2	-	×	256	72.84	2.11M	21.29B	4.20s	1.07s
2	V2	✓	×	256	72.56	1.15M	13.68B	3.23s	819ms
3	V2	✓	✓	256	72.97	1.02M	12.83B	3.16s	808ms
4	V2	✓	✓	128	72.74	0.98M	12.57B	3.12s	797ms
5	V3	-	×	256	72.64	3.60M	18.43B	4.17s	1.07s
6	V3	✓	×	256	71.91	1.76M	11.24B	3.01s	765ms
7	V3	✓	✓	256	72.37	1.63M	10.33B	2.96s	750ms
8	V3	✓	✓	128	72.36	1.51M	9.74B	2.87s	730ms
9	V2 0.5	✓	✓	128	68.57	0.28M	4.00B	1.61s	402ms
10	V2 0.35	✓	✓	128	66.83	0.16M	2.54B	1.31s	323ms
11	V3-Small	✓	✓	128	68.38	0.47M	2.90B	1.38s	349ms

Table 7. Semantic segmentation results on Cityscapes *val* set. **RF2**: Reduce the Filters in the last block by a factor of 2. V2 0.5 and V2 0.35 are MobileNetV2 with depth multiplier = 0.5 and 0.35, respectively. **SH**: Segmentation Head, where × employs the R-ASPP while ✓ employs the proposed LR-ASPP. **F**: Number of Filters used in the Segmentation Head. **CPU (f)**: CPU time measured on a single large core of Pixel 3 (floating point) w.r.t. a **full-resolution** input (i.e., 1024×2048). **CPU (h)**: CPU time measured w.r.t. a **half-resolution** input (i.e., 512×1024). Row 8, and 11 are our MobileNetV3 segmentation candidates. 知乎 @R.JD

Backbone	OS	mIOU	Madds (f)	Madds (h)	CPU (f)	CPU (h)
V3	16	72.6	9.74B	2.48B	2.87s	730ms
V3	32	72.0	7.74B	1.98B	2.39s	607ms
V3-Small	16	69.4	2.90B	0.74B	1.38s	349ms
V3-Small	32	68.3	2.06B	0.53B	1.16s	290ms
ESPNetv2 [32] †	-	62.1	13.46B	3.36B	-	-
CCC2 [34]	-	62.0	-	3.15B	-	-
ESPNetv1 [31] †	-	60.3	17.72B	4.43B	-	-
ESPNetv2 small [32] †	-	54.7	2.26B	0.56B	-	-

Table 8. Semantic segmentation results on Cityscapes test set. **OS**: Output Stride, the ratio of input image spatial resolution to backbone output resolution. When OS = 16, atrous convolution is applied in the last block of backbone. When OS = 32, no atrous convolution is used. **Madds (f)**: Multiply-Adds measured w.r.t. a **full-resolution** input (i.e., 1024×2048). **Madds (h)**: Multiply-Adds measured w.r.t. a **half-resolution** input (i.e., 512×1024). **CPU (f)**: CPU time measured on a single large core of Pixel 3 (floating point) w.r.t. a **full-resolution** input (i.e., 1024×2048). **CPU (h)**: CPU time measured w.r.t. a **half-resolution** input (i.e., 512×1024). †: The Madds are estimated from [32] which only provides Madds for input size 224×224 in their Fig. 7. 知乎 @R.JD

感觉实验结果没什么好说的。

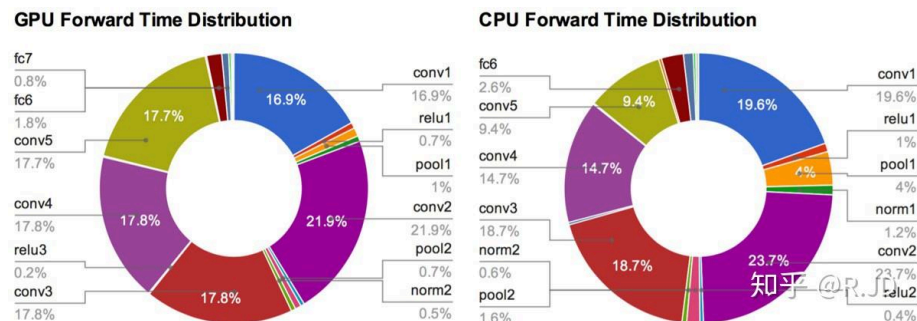
对了，有一点值得说一下，训练V3用的是4x4 TPU Pod，batch size 409...(留下了贫穷的泪水)

为什么MobileNet会这么快？

在写这篇文章的时候看到了一篇文章Why MobileNet and Its Variants (e.g. ShuffleNet) Are Fast?，这也让我有了一样的一个问题，这篇文章主要是从结构方面进行了讨论，从深度可分离卷积到组卷积的参数计算量等，因为之前的文章都有写过，在这里就不赘述了，感兴趣的同学可以翻阅下之前的文章。

在这里换一个角度。我们直接从**用时多少的角度**去讨论下这个问题。

下图来自Caffe作者贾扬清的博士论文：



该图是AlexNet网络中不同层的GPU和CPU的时间消耗，我们可以清晰的看到，不管是在GPU还是在CPU运行，最重要的“耗时杀手”就是conv，卷积层。也就是说，想要提高网络的运行速度，就得到提高卷积层的计算效率。

我们以MobileNetV1为主，看看MobileNet的资源分布情况：

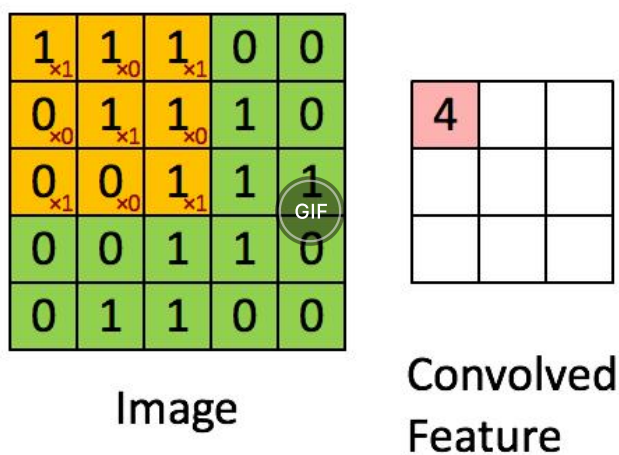
Table 2. Resource Per Layer Type

Type	Mult-Adds	Parameters
Conv 1×1	94.86%	74.59%
Conv DW 3×3	3.06%	1.06%
Conv 3×3	1.19%	0.02%
Fully Connected	0.18%	24.33%

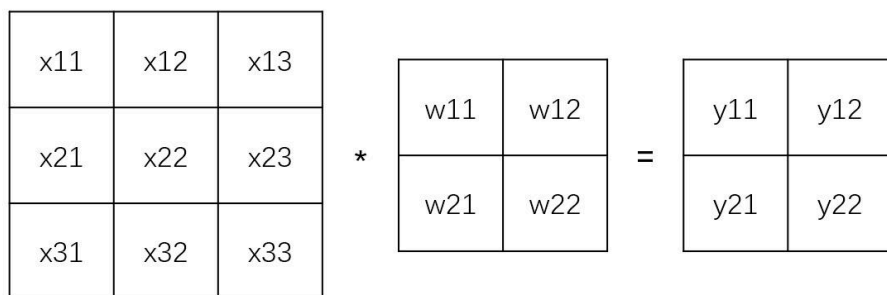
知乎 @R.JD

可以看到，**MobileNet**的95%的计算都花费在了 1×1 的卷积上，那 1×1 卷积有什么好处吗？

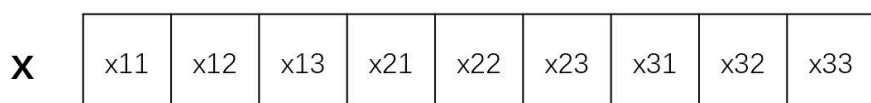
我们都知道，卷积操作就是如下图所示的乘加运算：



在计算机操作时，需要将其存入内存当中再操作（按照“行先序”）：

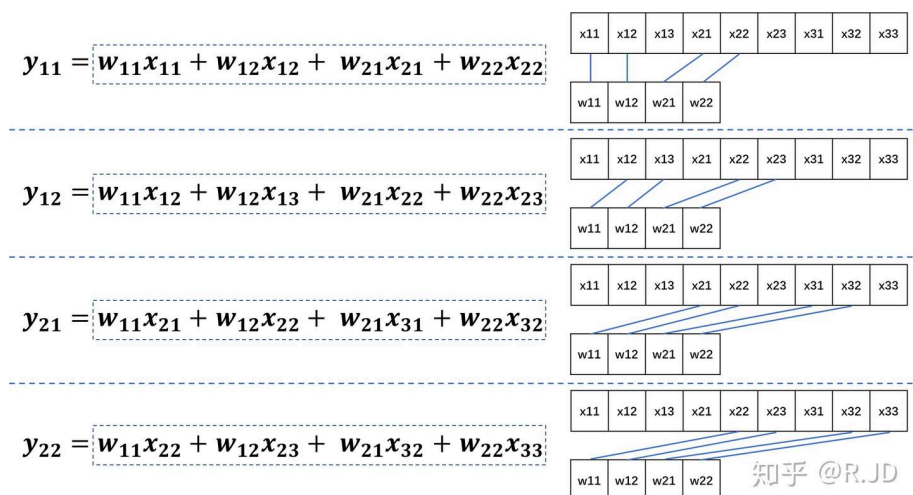


计算机内存 (“行先序”)



知乎 @R.JD

这样一来，特征图 $y_{11}, y_{12}, y_{21}, y_{22}$ 的计算如下所示：

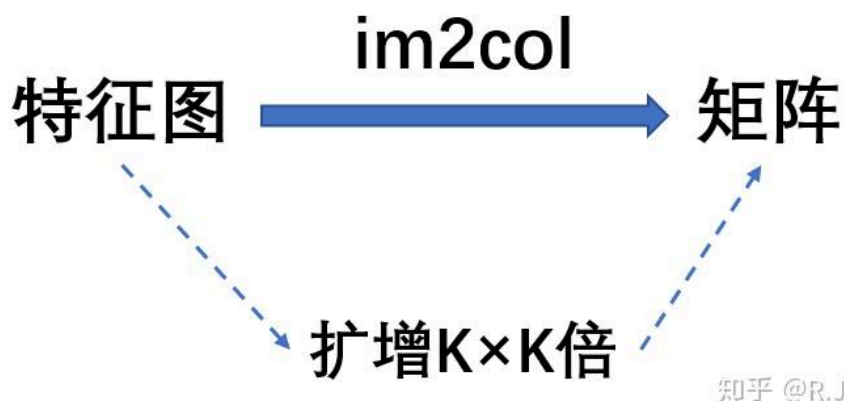


按照卷积计算，实线标注出卷积计算中的访存过程（对应数据相乘），我们可以看到这一过程是非常散乱和混乱的。直接用卷积的计算方式是比较愚蠢的。

这时候就要用到im2col操作。

im2col

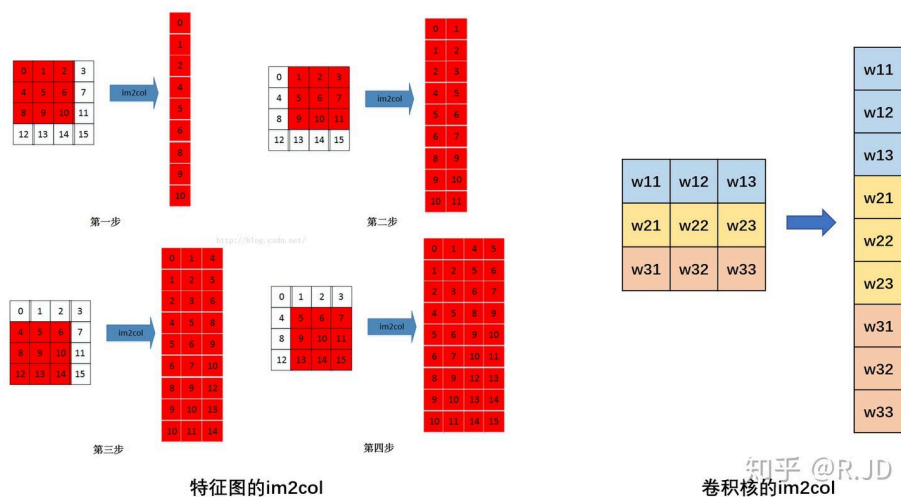
一句话来介绍im2col操作的话，就是通过牺牲空间的手段（约扩增 $K \times K$ 倍），将特征图转换成庞大的矩阵来进行卷积计算。



其实思路非常简单：

把每一次循环所需要的数据都排列成列向量，然后逐一堆叠起来形成矩阵（按通道顺序在列方向上拼接矩阵）。

比如 $C_i \times W_i \times H_i$ 大小的输入特征图， $K \times K$ 大小的卷积核，输出大小为 $C_o \times W_o \times H_o$ ，输入特征图将按需求被转换成 $(K \times K) \times (C_i \times W_o \times H_o)$ 的矩阵，卷积核将被转换成 $C_o \times (K \times K)$ 的矩阵，



然后调用**GEMM（矩阵乘矩阵）**库加速两矩阵相乘也就完成了卷积计算。由于**按照计算需求排布了数据顺序**，每次计算过程中总是能够依次访问特征图数据，极大地提高了计算卷积的速度。（不光有GEMM，还有FFT（快速傅氏变换））



换一种表示方法能更好地理解，图片来自[High Performance Convolutional Neural Networks for Document Processing](#)：

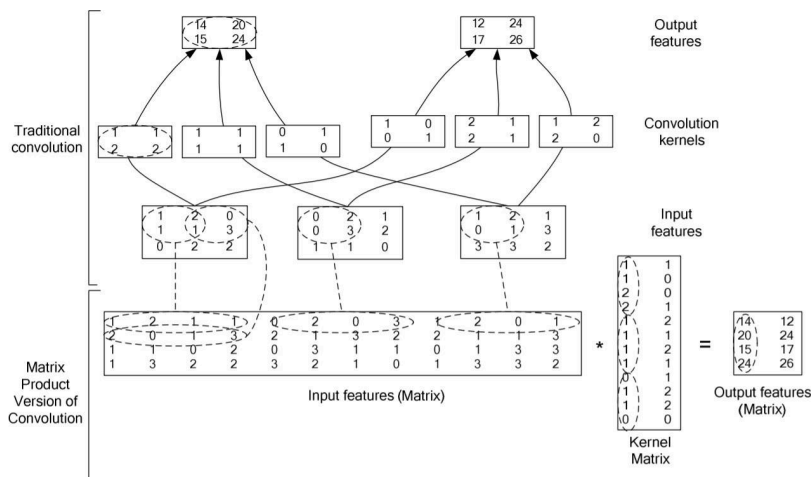
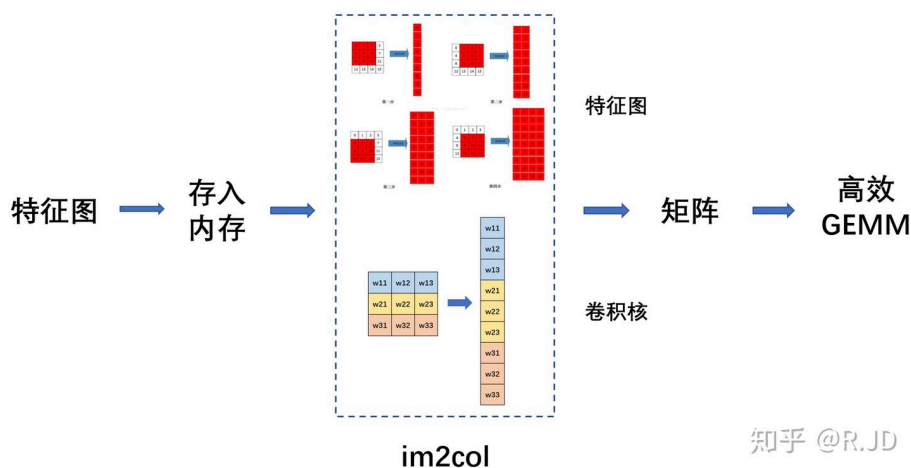


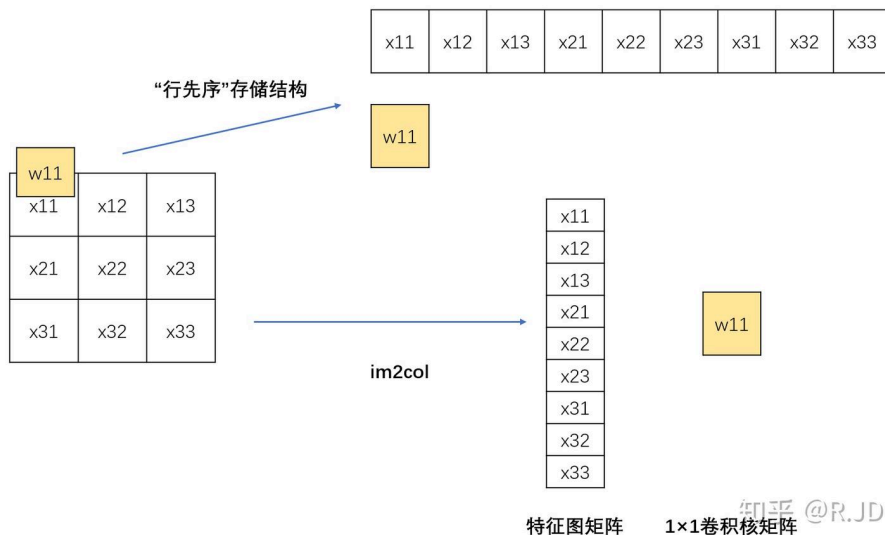
Figure 2. Example convolution operations in a convolutional layer (biases, sub-sampling, and non-linearity omitted). The top figure presents the traditional convolution operations, while the bottom figure presents the matrix version.

这样可以更清楚的看到卷积的定义进行卷积操作（上图上半部分），内存访问会非常不规律，以至于性能会非常糟糕。而`im2col()`以一种内存访问规则的方式排列数据，虽然`im2col`操作增加了很多数据冗余，但使用`Gemm`的性能优势超过了这个数据冗余的劣势。

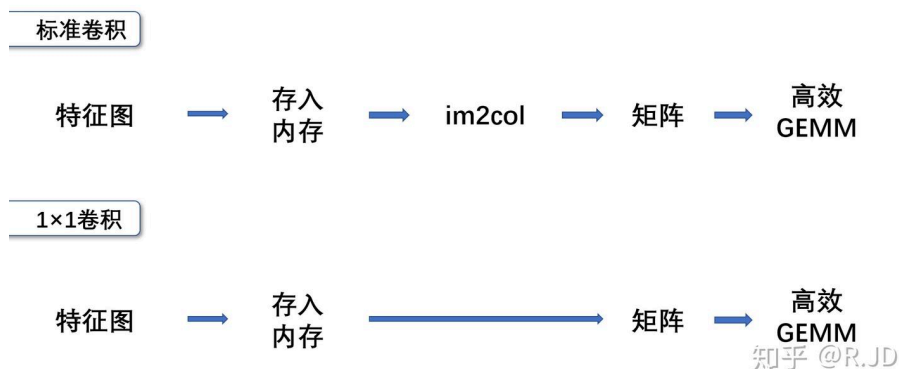
所以标准卷积运算大概就是这样的一个过程：



那我们现在回到 1×1 的卷积上来，有点特殊。按照我们之前所说的， 1×1 的卷积的原始储存结构和进行`im2col`的结构如下图所示：



可以看到矩阵是完全相同的。标准卷积运算和 1×1 卷积运算对比如下图：



也就是说，**1x1卷积不需要im2col的过程**，所以底层可以有更快的实现，拿起来就能直接算，大大节省了数据重排列的时间和空间。

当然，这些也不是那么绝对的，因为毕竟MobileNet速度快不快，与CONV1x1运算的优化程度密切相关。如果使用了定制化的硬件（比如用FPGA直接实现3x3的卷积运算单元），那么im2col就失去了意义而且反而增加了开销。

回到之前的MobileNet的资源分布，95%的 1×1 卷积和优化的网络结构就是MobileNet能如此快的原因了。

Table 2. Resource Per Layer Type

Type	Mult-Adds	Parameters
Conv 1×1	94.86%	74.59%
Conv DW 3×3	3.06%	1.06%
Conv 3×3	1.19%	0.02%
Fully Connected	0.18%	24.33%

Reference

[MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications](#)

[MobileNetV2: Inverted Residuals and Linear Bottlenecks](#)

[Searching for MobileNetV3](#)

[Xception: Deep Learning with Depthwise Separable Convolutions](#)

[Simplifying ConvNets for Fast Learning](#)

[a-basic-introduction-to-separable-convolution](#)

[CNN模型之MobileNet](#)

[网络解析（二）：MoblieNets详解](#)

[轻量级网络--MobileNet论文解读](#)

mp.weixin.qq.com/s/O2Bh...

machinethink.net/blog/m...

[如何评价 Google 最新的模型 MnasNet? - 曲晓峰的回答 - 知乎](#)

[Learning Semantic Image Representations at a Large Scale 贾扬清博士论文](#)

[im2col的原理和实现](#)

[在 Caffe 中如何计算卷积? - 贾扬清的回答 - 知乎](#)

[漫谈卷积层](#)

[High Performance Convolutional Neural Networks for Document Processing](#)

[Why MobileNet and Its Variants \(e.g. ShuffleNet\) Are Fast](#)

所属专栏 · 2020-11-04 22:25 更新



深度学习需要深度学习

R.JD

6 篇内容 · 2577 赞同

订阅

最热内容 · 轻量级神经网络“巡礼”（一）—— ShuffleNetV2

发布于 2019-06-26 09:19

107 条评论

默认

最新



baihaojie

" $28 \times 28 \times 32$ 那一层的步长为2的话，输出应该是 14×14 ，应该是一处错误。按照作者论文里的说法，自己修改了一下："这个是不是原文是对的啊？

2019-06-29

回复 25



蓝熊 · 范欢喜、

原文是对的。。。刚检查了一下。
如果改的话，后面会变成 $28 \times 28 \times 32$ 会变成 $28 \times 28 \times 64$

2020-09-07

回复 2



范欢喜、

我也觉得原文是对的

2020-01-09

回复 1



海盗船长

对于标准卷积和深度卷积过程对比的图是否存在错误？图中深度可分离是 $12 \times 13 \times 3$ ---
($5 \times 5 \times 1 \times 1$)--($1 \times 1 \times 3 \times 256$)--> $12 \times 12 \times 256$ 。应该为 $12 \times 13 \times 3$ ---($5 \times 5 \times 1 \times 3$)--($1 \times 1 \times 3 \times 256$)--
> $12 \times 12 \times 256$ 。

2020-03-01

回复 24



旺角海门

我觉得也应该是 $5 \times 5 \times 1 \times 3$

2020-03-21

回复 2



知乎用户QJhWQi

我也这么认为，因为后边给出的公式确实是 $D \times D \times M$

2020-07-10

回复 1



知乎用户AWq6wj

v3中“sigmoid 非常明智的选择”，加个“不”呀



2019-11-18

● 回复 ♥ 12



咖发咖瓦

...

看了一下V1和V2的论文，然后再去看官方源码，激活函数V1的时候还是用RELU的，到了V2才用的RELU6

2022-09-22

● 回复 ♥ 5



三年不改名

...

写的很棒，对V1和V2有了更好的理解。

2019-08-19

● 回复 ♥ 5



Hotcooky

...

楼主的思路和表达能力都很强

2019-11-27

● 回复 ♥ 4



李森森

...

赞

2021-11-06

● 回复 ♥ 3



Domadaaa

...

想问一下为什么讲V3的h-swish函数的时候，“随着网络的深入，应用非线性激活函数的成本会降低，能够更好的减少参数量”呀，怎么理解

2023-06-06

● 回复 ♥ 1



PaulSt

...

应该是256个filter

2021-08-27

● 回复 ♥ 1



菜包

...

什么是线性激活函数

2021-02-25

● 回复 ♥ 1

[点击查看全部评论 >](#)